

CS4231
Parallel and Distributed Algorithms

Lecture 10
Instructor: Haifeng YU

Review of Last Lecture

<u>System/Failure Model</u>	<u>Consensus Protocol</u>
Ver 0: No node or link failures	Trivial: all-to-all broadcast
Ver 1: Node crash failures; Channels are reliable; Synchronous;	$(f+1)$ -round protocol can tolerate f crash failures
Ver 2: No node failures; Channels may drop messages (the coordinated attack problem)	Impossible without error Randomized algorithm with $1/r$ error prob
Ver 3: Node crash failures; Channels are reliable; Asynchronous;	Impossible (the FLP theorem)
Ver 4: Node Byzantine failures; Channels are reliable; Synchronous; (the Byzantine Generals problem)	If $n \leq 3f$, impossible. If $n \geq 4f + 1$, we have a $(2f+2)$ -round protocol.

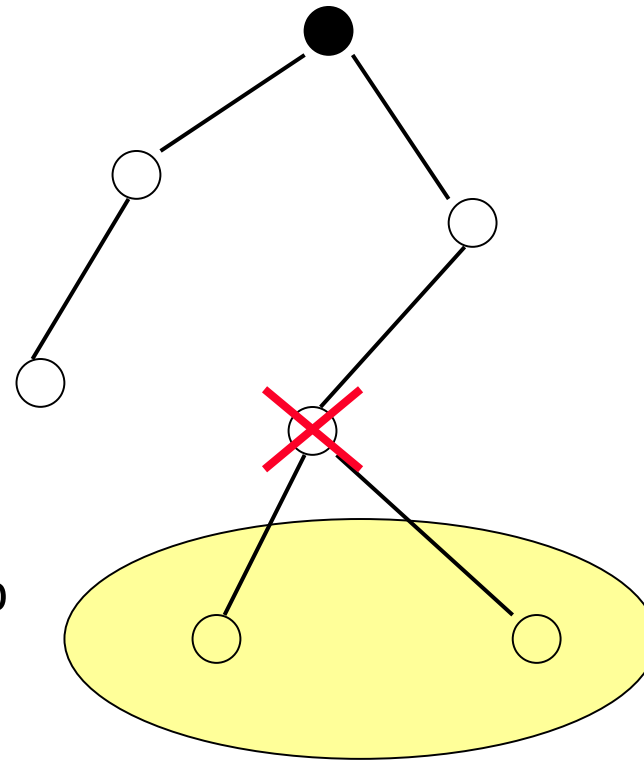
Today's Roadmap

- “Self-Stabilization”
- Formalizing the notion of self-stabilization
- Rotating-privilege problem
- Self-Stabilizing Spanning Tree Construction
 - Very useful in multicast (BitTorrent-style data streaming)

Motivation for Self-Stabilization

- Distributed systems can get into illegal state due to
 - Topology changes
 - Failures / reboots
 - Generally called “faults”

A multicast tree: Each node records who its parent and children are

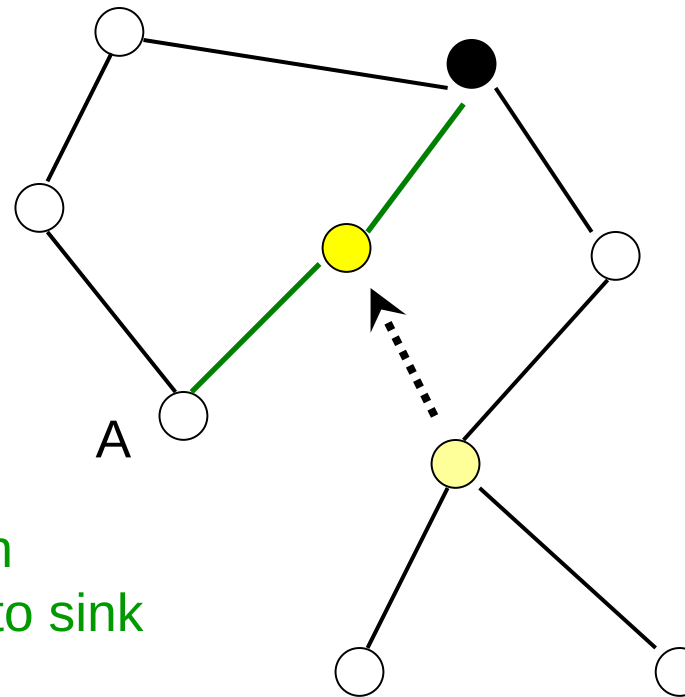


parent value of these two nodes no longer valid

Motivation for Self-Stabilization

- Distributed systems can get into illegal state due to
 - Topology changes
 - Failures / reboots
 - Generally called “faults”

Mobile ad hoc networks:
Maintaining the shortest route back to sink



A should now have an improved route back to sink

Defining Self-Stabilization

- The state (i.e., data state in all processes) of a distributed system is either legal or illegal
 - Definition based on application semantics
 - The code on each process is assumed to be correct all the time
- A distributed algorithm is self-stabilizing if
 - Starting from any (legal or illegal) state, the protocol will eventually reach a legal state if there are no more faults
 - Once the system is in a legal state, it will only transit to other legal states unless there are faults
- Intuitively, will always recover from faults and once recovered, will stay recovered forever
- Self-stabilizing algorithm typically runs in background and never stops

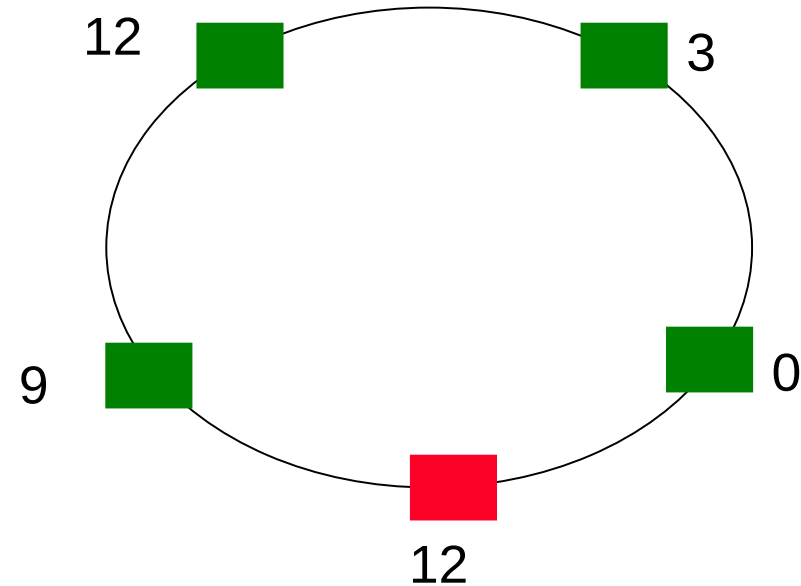
The Rotating Privilege Problem

- A ring of n processes, each process can only communicate with neighbors
- There is a privilege in the system
 - At any time, only one node may have the privilege (you can think of this as a token)
 - The node with the privilege may for example, have exclusive access to some resource
 - The privilege needs to “rotate” among the nodes so that each node has a chance

The Rotating Privilege Algorithm

- Each process i has a local integer variable V_i
 - $0 \leq V_i \leq k$ where k is some constant no smaller than n

Example: $n = 5$ and $k = 12$



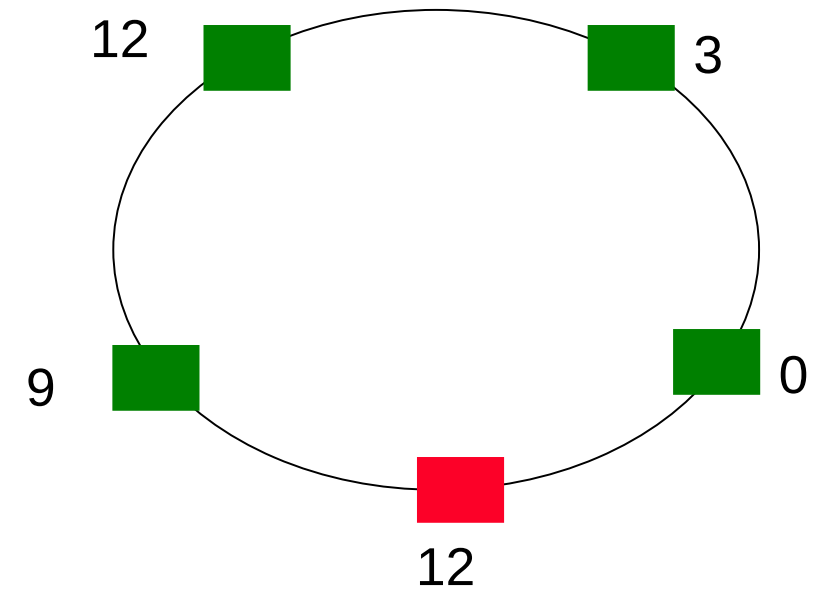
Red process's action:

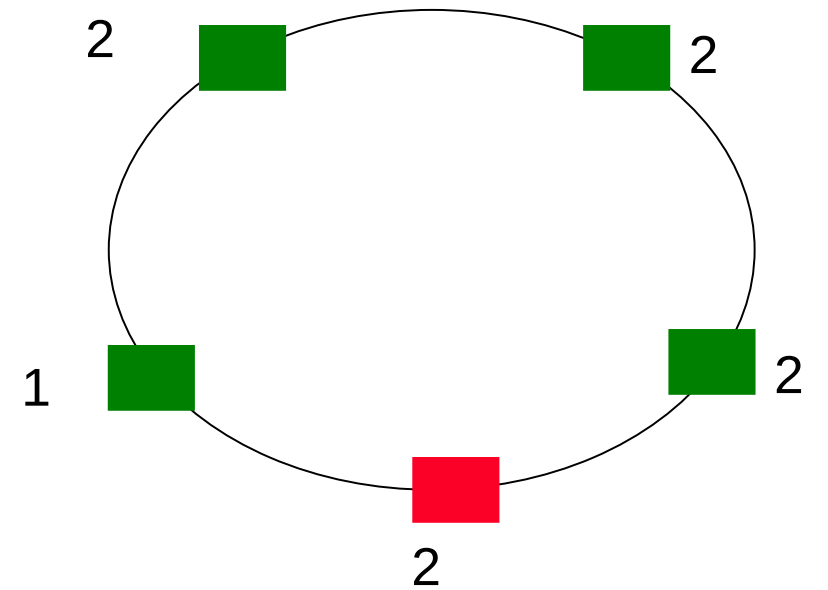
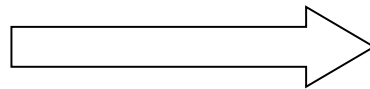
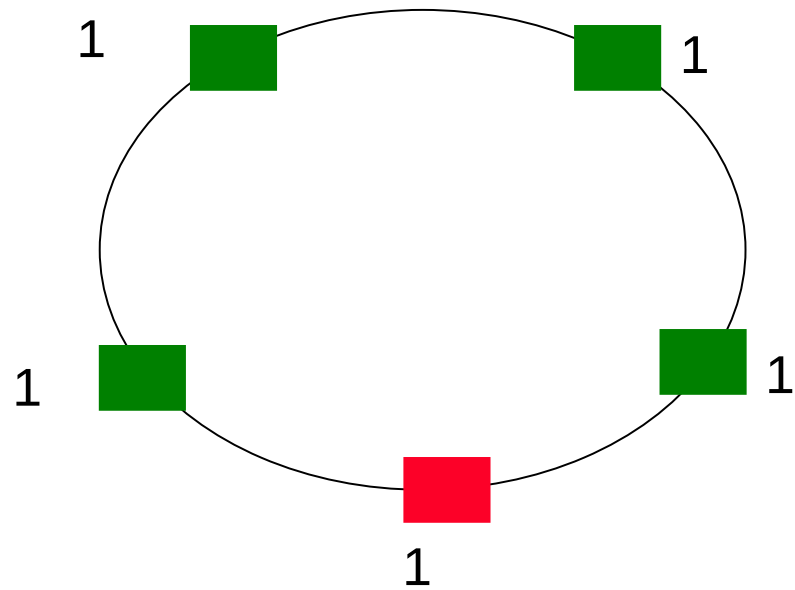
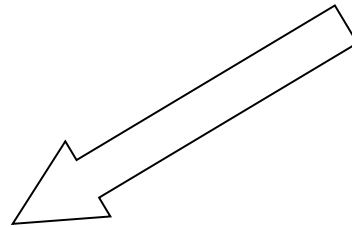
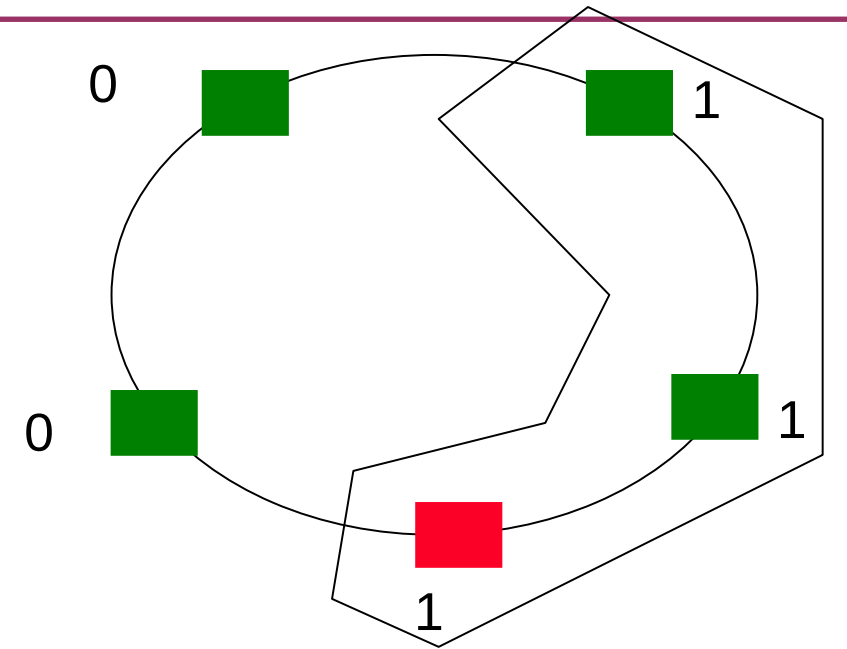
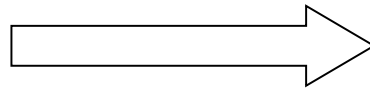
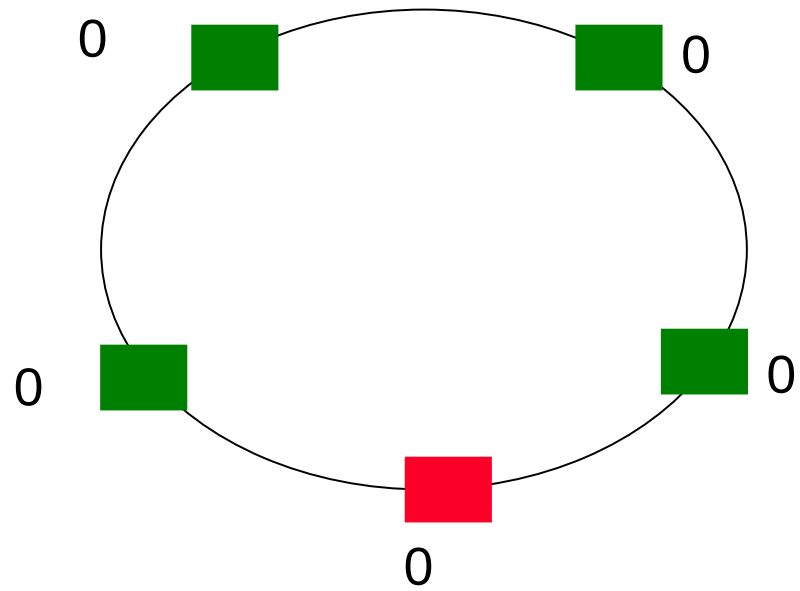
```
Retrieve value L of my clockwise neighbor;  
Let V be my value;  
if (V == L) { // I have the privilege  
    // complete whatever I want to do;  
    V = (V+1) % k;  
}
```

Green process's action:

```
Retrieve value L of my clockwise neighbor;  
Let V be my value;  
if (V != L) { // I have the privilege  
    // complete whatever I want to do;  
    V = L;  
}
```

Each process executes each action repeatedly – we will assume each action happens instantaneously and no interleaving among actions (for this algorithm only)





What's Interesting about the Algorithm

- This problem is mainly for theoretical interests
- What is interesting about it:
 - Regardless of the initial values of the processes, eventually the system will get into a legal state and stay in legal states
 - Self-stabilizing!

Legal States

- We say that a process makes a “move” if it has the privilege and changes its value
- System in legal state if exactly one process can make a move
- **Lemma: The following are legal states and are the only legal state**
 - All n values same OR
 - Only two different values forming two consecutive bands, and one band starts from the red process
- To prove these are legal states, only need to confirm there is exactly one process that can make a move
- To prove these are the **only** legal states, consider the value V of the red process and the value L of its clockwise neighbor
 - Case I: $V=L$. Then the red process can make a move. No other process should be able to make a move. Hence all n values must be same.
 - Case II: $V \neq L$. Starting from the red process, find counter-clockwise the first green process whose value is different from its clockwise neighbor. Such green process must exist since $V \neq L$. This green process can make a move. No other process should be able to make a move. Hence the two bands...

Legal States $\Rightarrow$ Legal States

- **Theorem: If the system is in a legal state, then it will stay in legal states**
 - Our assumption on instantaneous actions will simplify this proof.
 - We can all possible actions.
 - For the red process, the only action that can change the system state happens when $V=L$, and that action will update V to be $(V+1) \% k$.
 - As we show earlier, when $V=L$, the only legal state is for all n values to be the same. Hence updating V to be $(V+1) \% k$ will result in two bands of values, which is also a legal state.
 - For the green process, the only action that can change the system state happens when $V \neq L$, and that action will update V to L .
 - As we show earlier, when $V \neq L$, the only legal state is to have two bands of values. Updating V to be L will either result in all n values being the same (if the green process is the clockwise neighbor of the red process), or still result in two bands of values (if otherwise). In either case, the system state is still legal.

Illegal States => Legal States

- Lemma 1: Let P be a green process, and let Q be P's clockwise neighbor. (Q can be either green or red.) If Q makes i moves, then P can make at most i+1 move.
- Lemma 2: Let Q be the red process. If Q makes i moves, then system-wide there can be at most the following number of moves:

$$i + (i + 1) + (i + 2) + \dots + (i + n - 1) = ni + \frac{(n - 1)n}{2}$$

- Lemma 3: Consider a sequence of $(n^2 - n)/2 + 1$ moves in the system. The red process makes at least one move in the sequence.
- Lemma 4: In any system state (regardless of legal or not), there is always at least one process that can make a move.
 - Consider the value V of the red process and the value L of its clockwise neighbor. If $V = L$, the red process can make a move. If $V \neq L$, there must exist some green process whose value differs from its clockwise neighbor's. That green process can make a move.

Illegal States => Legal States

- Lemma 5: Regardless of the starting state, the system eventually reach a state T where the red process has a different value from the values of any other process (though the system may not remain in such states forever)
 - Proof: Let Q be the red process. If in the starting state Q has the same value as some other process, then there must be an integer j ($0 \leq j \leq k-1$) that is not the value of any process.
 - In any state, at least one process can make move
 - Eventually, the number of moves will approach infinity
 - Q moves once among every consecutive $(n^2-n)/2+1$ moves of the system
 - Q will make infinite number of moves
 - Q will eventually take j as its value. (It takes Q any most k moves to do so.)

Illegal States => Legal States

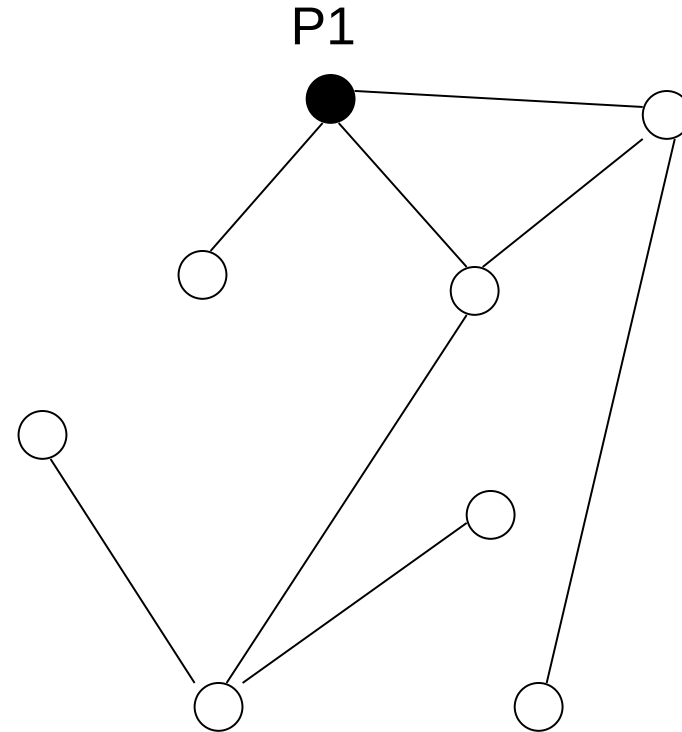
- Lemma 6: If the system is in a state where the red process has a different value from all other process, then the system will eventually reach another state where all processes have the same value (though the system make not remain in such a state forever).
 - Proof: Let the red process be p_1 , and let p_1 's value be x . Starting from the red process and counter-clockwise along the ring, let the green processes be $p_2, p_3, \dots, p_n$.
 - Let t be such that p_1 through p_t all have values of x , and p_{t+1} through p_n all have values different from x . Initially $t=1$. We will prove that for any $t = s$, t will become $s+1$ at some point of time. Hence eventually t will be n , and we are done.
 - To prove the above claim, note that p_{s+1} will take the value of x once it takes an action. Denote this event as E . Before event E , p_{s+2} through p_n can never take the value of x . Also, since p_n can never take the value of x before event E , p_1 through p_s can never change their value before event E . Hence, immediately after event E happens, t becomes $s+1$.

Illegal States $\Rightarrow$ Legal States

- Theorem: Regardless of the initial states of the system, the system will eventually reach a legal state.
 - Proof: From Lemma 5 and Lemma 6.

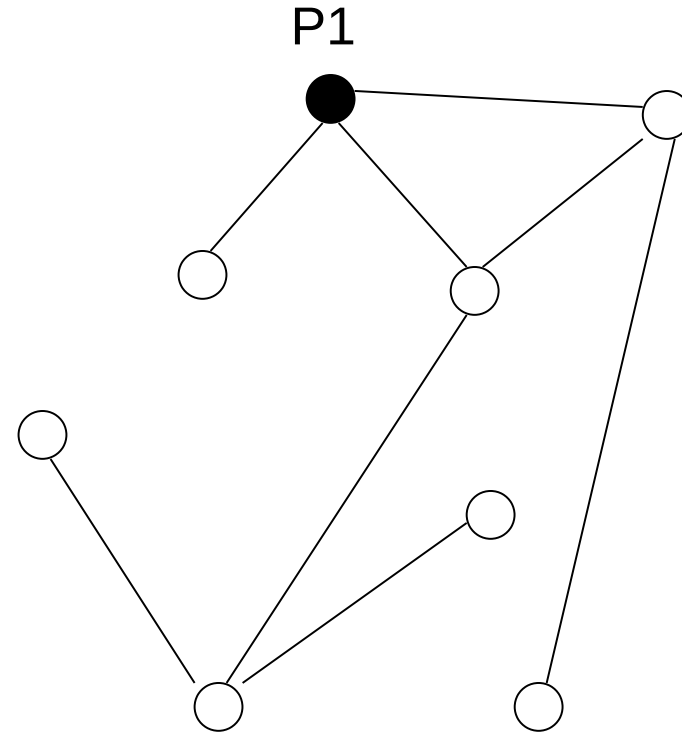
Self Stabilizing Spanning Tree Algorithm

- Given n processes connected by an undirected graph and one special process $P1$, construct a spanning tree rooted at $P1$
 - Not all processes can communicate with all processes directly
- A very useful / practical algorithm
 - Can also be used to compute shortest path



Self Stabilizing Spanning Tree Algorithm

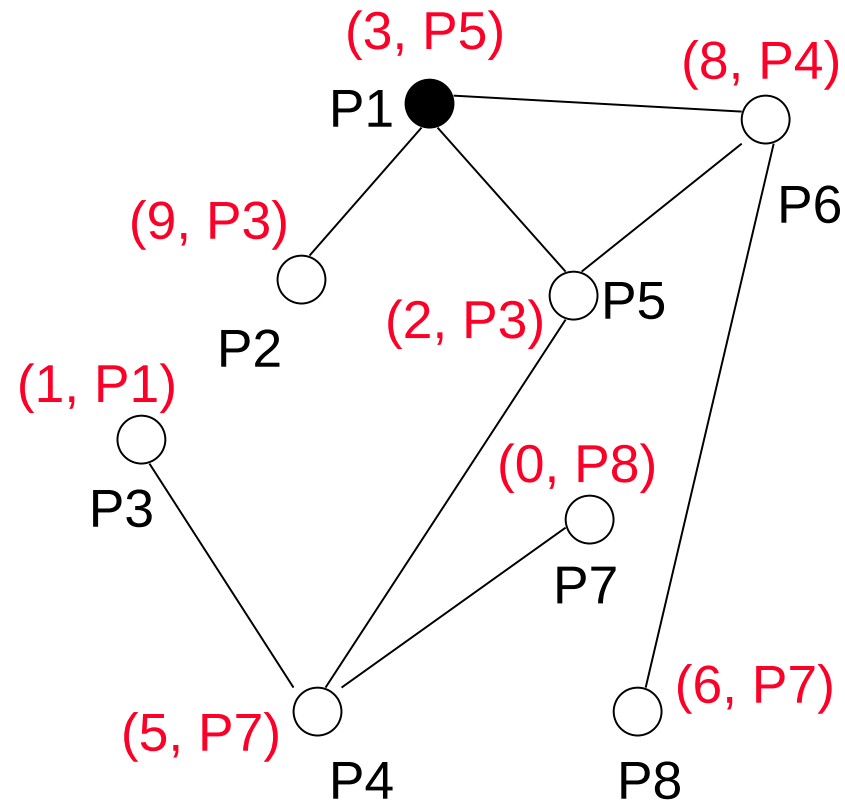
- Each process maintains two variables
 - **parent**: Who my parent is
 - **dist**: My distance to root (dist not allowed to be negative)
- Runs in the background
 - **parent** and **dist** are continuously updated
- At any given point of time, the values of the two variables can be wrong
 - Due to “faults” such as topology change resulted from node movement



Self Stabilizing Spanning Tree Algorithm

- On P1 (executed periodically):
 - $\text{dist} = 0$; $\text{parent} = \text{null}$;
- On all other processes (executed periodically):
 - Retrieve dist from all neighbors
 - Set my own $\text{dist} = 1 + (\text{the smallest } \text{dist} \text{ received})$
 - Set my own $\text{parent} = \text{my neighbor with the smallest } \text{dist}$ (with tie breaking if needed)

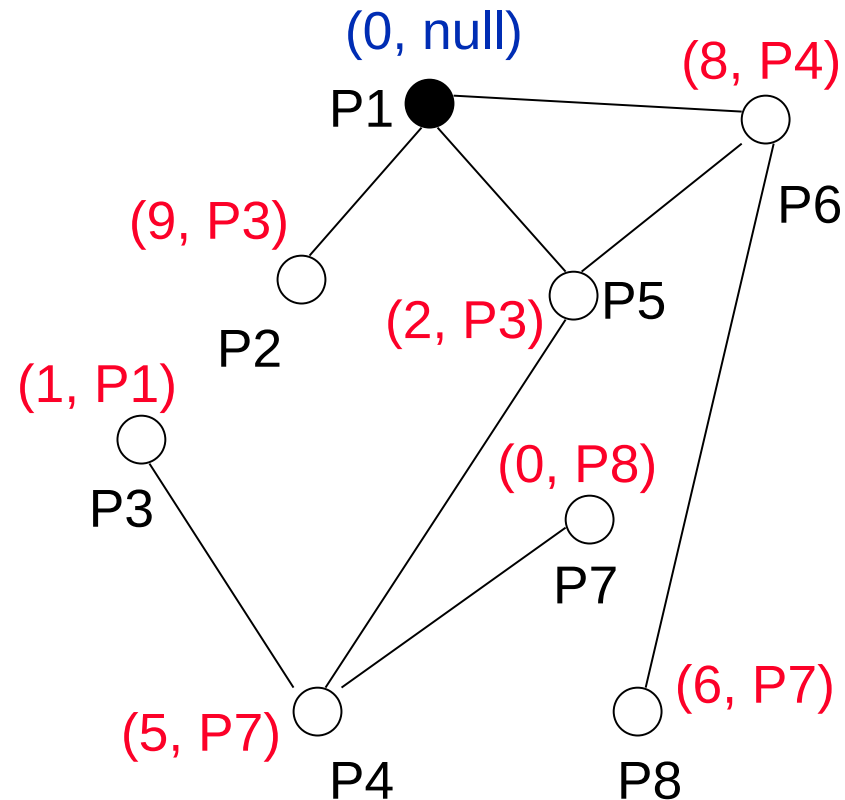
Red values are initially incorrect values;
blue values are values that have become correct



Self Stabilizing Spanning Tree Algorithm

- On P1 (executed periodically):
 - $\text{dist} = 0$; $\text{parent} = \text{null}$;
- On all other processes (executed periodically):
 - Retrieve dist from all neighbors
 - Set my own $\text{dist} = 1 + (\text{the smallest } \text{dist} \text{ received})$
 - Set my own $\text{parent} = \text{my neighbor with the smallest } \text{dist}$ (with tie breaking if needed)

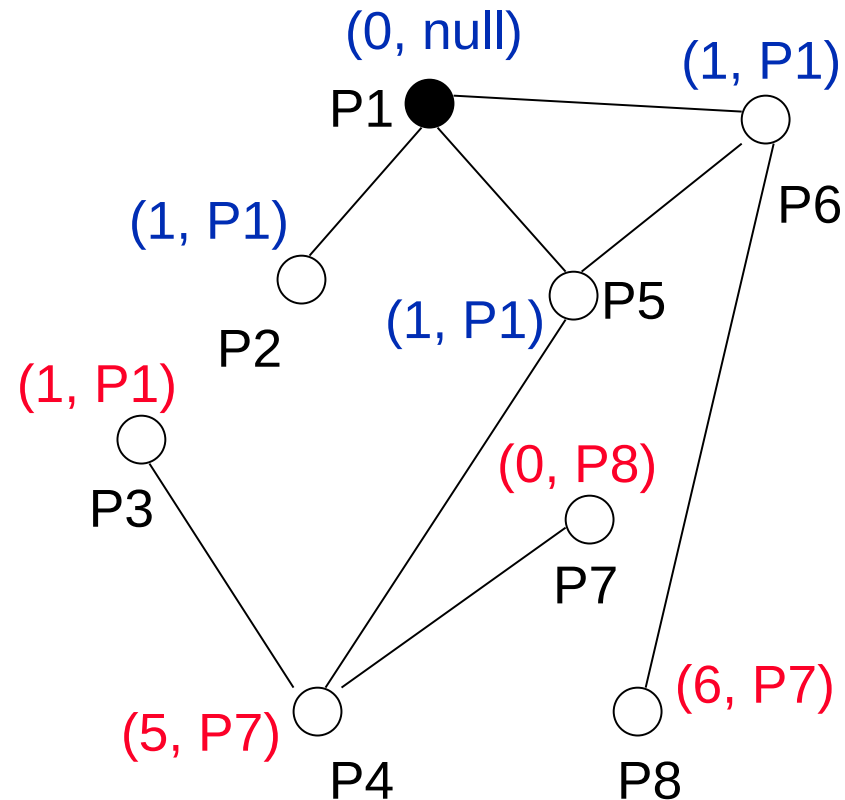
Red values are initially incorrect values;
blue values are values that have become correct



Self Stabilizing Spanning Tree Algorithm

- On P1 (executed periodically):
 - $\text{dist} = 0$; $\text{parent} = \text{null}$;
- On all other processes (executed periodically):
 - Retrieve dist from all neighbors
 - Set my own $\text{dist} = 1 + (\text{the smallest } \text{dist} \text{ received})$
 - Set my own $\text{parent} = \text{my neighbor with the smallest } \text{dist}$ (with tie breaking if needed)

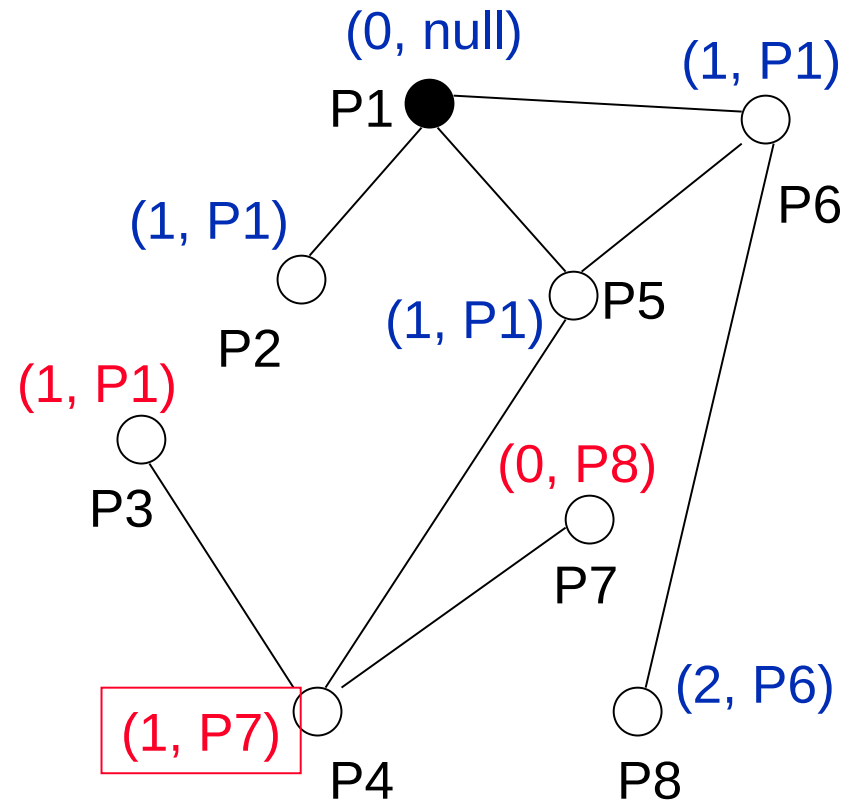
Red values are initially incorrect values;
blue values are values that have become correct



Self Stabilizing Spanning Tree Algorithm

- On P1 (executed periodically):
 - $\text{dist} = 0$; $\text{parent} = \text{null}$;
- On all other processes (executed periodically):
 - Retrieve dist from all neighbors
 - Set my own $\text{dist} = 1 + (\text{the smallest } \text{dist} \text{ received})$
 - Set my own $\text{parent} = \text{my neighbor with the smallest } \text{dist}$ (with tie breaking if needed)

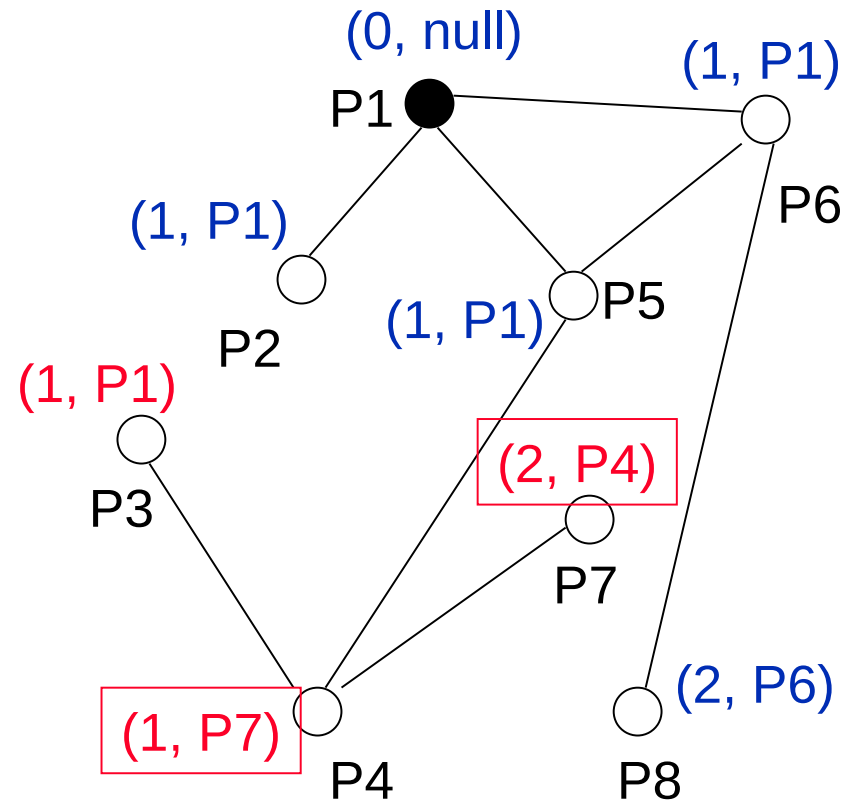
Red values are initially incorrect values;
blue values are values that have become correct



Self Stabilizing Spanning Tree Algorithm

Red values are initially incorrect values;
blue values are values that have
become correct

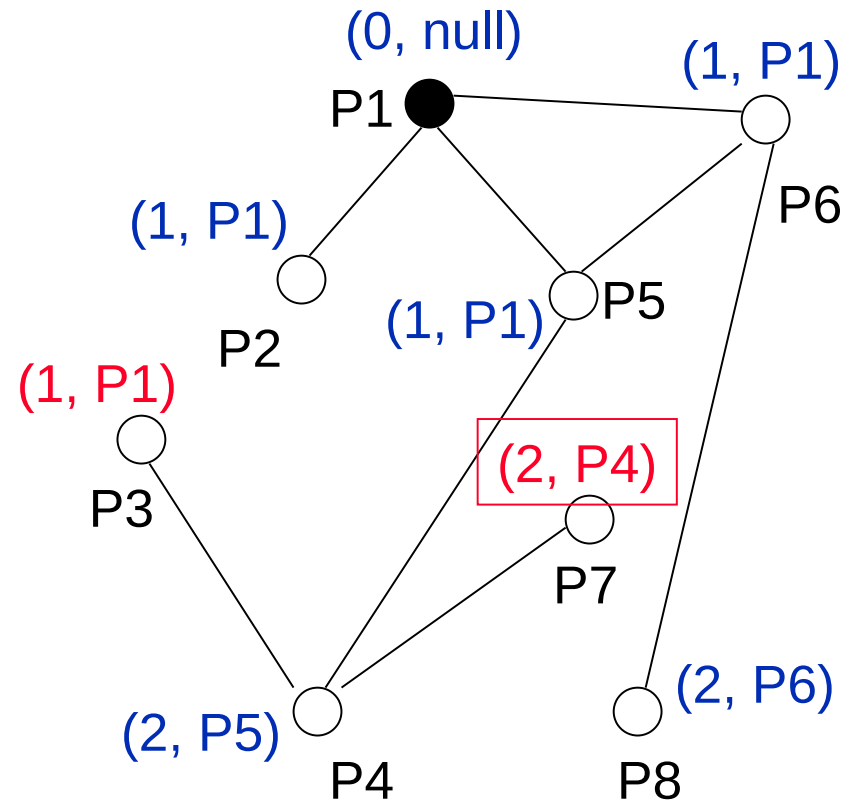
- On P1 (executed periodically):
 - $\text{dist} = 0$; $\text{parent} = \text{null}$;
- On all other processes (executed periodically):
 - Retrieve dist from all neighbors
 - Set my own $\text{dist} = 1 + (\text{the smallest } \text{dist} \text{ received})$
 - Set my own $\text{parent} = \text{my neighbor with the smallest } \text{dist}$ (with tie breaking if needed)



Self Stabilizing Spanning Tree Algorithm

- On P1 (executed periodically):
 - $\text{dist} = 0$; $\text{parent} = \text{null}$;
- On all other processes (executed periodically):
 - Retrieve dist from all neighbors
 - Set my own $\text{dist} = 1 + (\text{the smallest } \text{dist} \text{ received})$
 - Set my own $\text{parent} = \text{my neighbor with the smallest } \text{dist}$ (with tie breaking if needed)

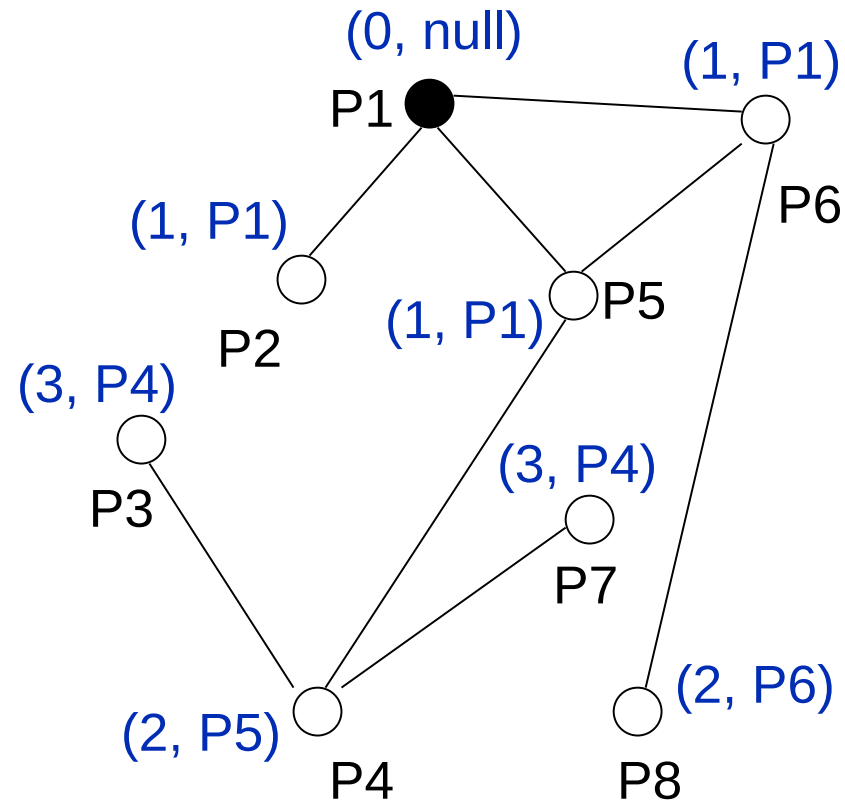
Red values are initially incorrect values;
blue values are values that have become correct



Self Stabilizing Spanning Tree Algorithm

- On P1 (executed periodically):
 - $\text{dist} = 0$; $\text{parent} = \text{null}$;
- On all other processes (executed periodically):
 - Retrieve dist from all neighbors
 - Set my own $\text{dist} = 1 + (\text{the smallest } \text{dist} \text{ received})$
 - Set my own $\text{parent} = \text{my neighbor with the smallest } \text{dist}$ (with tie breaking if needed)

Red values are initially incorrect values;
blue values are values that have become correct



A side note: We have learned two spanning tree construction algorithms so far...

- Self Stabilizing Spanning Tree Algorithm in this lecture
- Spanning tree construction algorithm that we learned for computing the number of nodes (in the context of leader election)
- How do these two algorithms differ?

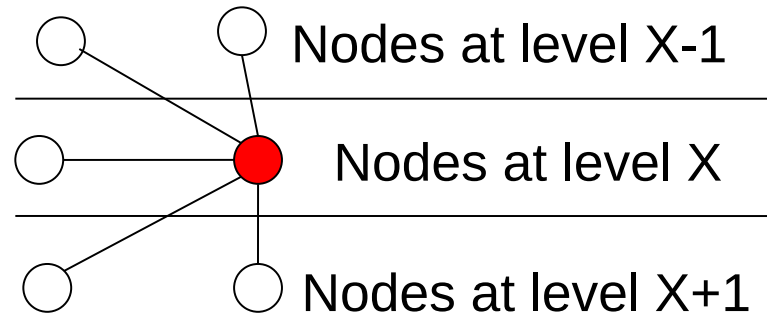
Quick Poll <https://pollev.com/haifengyu229>

Correctness Proof

- Define a phase to be the minimum time period where each process has executed its code at least once (called “has taken an action”)
 - Some process may execute its code more than once
 - The definition of a phase here is different from a round in synchronous systems !
- Let A_i to be the length of the shortest path from process i to the root, let $dist_i$ to be the value of **dist** on process i
 - A_i is called the **level** of process i
 - When the topology does not change: A_i is fixed and $dist_i$ may change.

Properties of the Levels of the Nodes

1. A node at level X has at least one neighbor in level $X-1$.
2. A node at level X can only have neighbors in level $X-1$, X , and $X+1$.



Correctness Proof

- Lemma: At the end of phase 1, $\text{dist}_1 = 0$ and $\text{dist}_i \geq 1$ for all $i \geq 2$
- Lemma: At the end of phase r ,
 - Any process i whose $A_i \leq r-1$, we have $\text{dist}_i = A_i$;
 - Any process i whose $A_i \geq r$, we have $\text{dist}_i \geq r$;

Correctness Proof

- Prove by induction: The induction base already holds for $r = 1$.
- Assume the lemma holds at phase r , now consider phase $r+1$. Namely, by inductive hypothesis, we already know that at the end of phase r :
 - Any process i whose $A_i \leq r-1$, we have $\text{dist}_i = A_i$;
 - Any process i whose $A_i \geq r$, we have $\text{dist}_i \geq r$;
- And we need to prove that at the end of phase $r+1$:
 - Any process i whose $A_i \leq r$, we have $\text{dist}_i = A_i$;
 - Any process i whose $A_i \geq r+1$, we have $\text{dist}_i \geq r+1$;

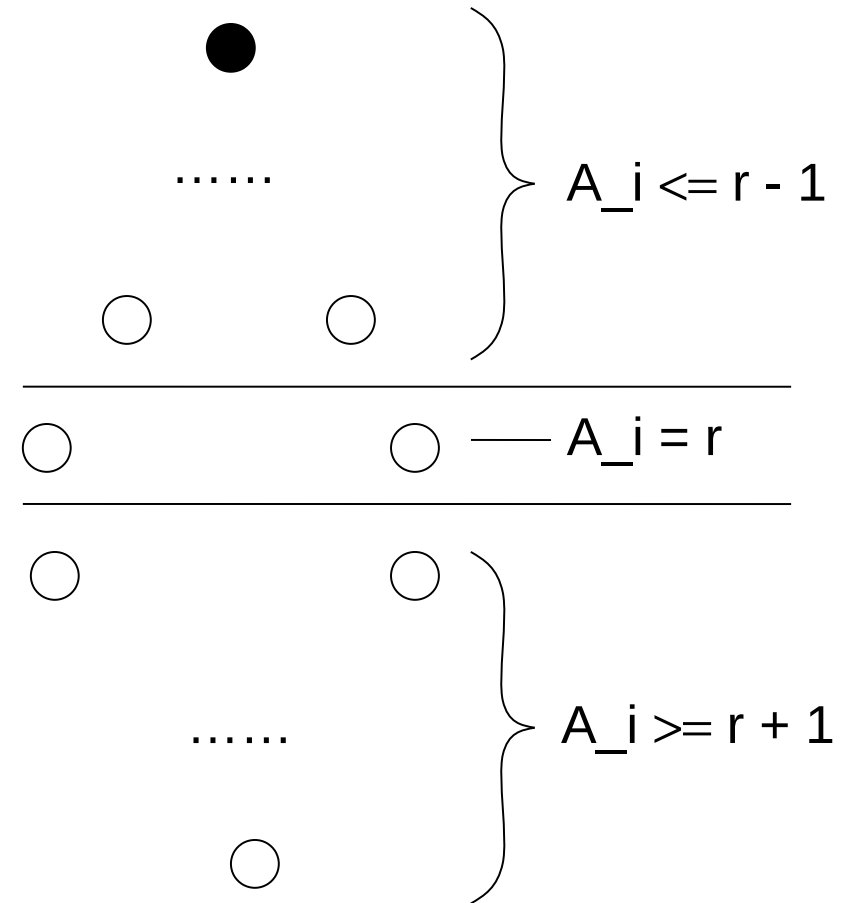
Correctness Proof

- Consider all t actions taken during phase $r+1$
 - We will use an induction on t
- This proof is tricky because:
 - A process may take multiple actions in a phase !
 - Processes may take actions in parallel – cannot assume a serialization of all actions !
- The proof technique is typical for proving self-stabilization
 - Step 1: Prove that the t actions will not roll back what is already achieved so far by phase r (no backward move)
 - Step 2: Prove that at some point, each node will achieve more (forward move)
 - Step 3: Prove that the t actions will not roll back the effects of the forward move after the forward move happens (no backward move after the forward move)

Claim 1: The blue conditions hold *throughout* phase $r+1$.

Proof: Use an induction on t . If $t = 0$, trivial. Next assume the statement holds for t and consider action $(t+1)$ by some node i . (Cannot assume that action $(t+1)$ happens after the t actions.)

for nodes with	already know: phase r	want to show: phase $r+1$
$A_i \leq r-1$	$\text{dist}_i = A_i$	$\text{dist}_i = A_i$
$A_i = r$	$\text{dist}_i \geq r$	$\text{dist}_i = r$
$A_i \geq r+1$	$\text{dist}_i \geq r$	$\text{dist}_i \geq r+1$

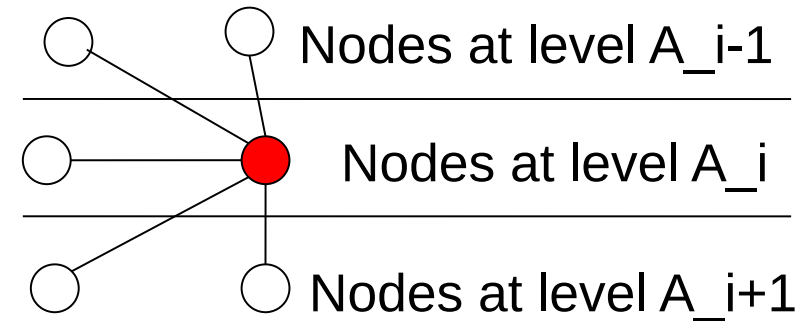


Proof (continued):

Case I: $A_i \leq r-1$. Node i has at least one neighbor Y at level A_i-1 . By the blue condition and by our inductive hypothesis, we know that Y 's dist value is exactly A_i-1 . Furthermore, node i only has neighbors at level A_i-1 , A_i , and A_i+1 . By the blue condition and by our inductive hypothesis, the dist value on any of these neighbors will not be smaller than A_i-1 . Hence the min dist seen by node i is A_i-1 . Hence node i will set its dist value to be A_i .

Case II: $A_i \geq r$. Similar...

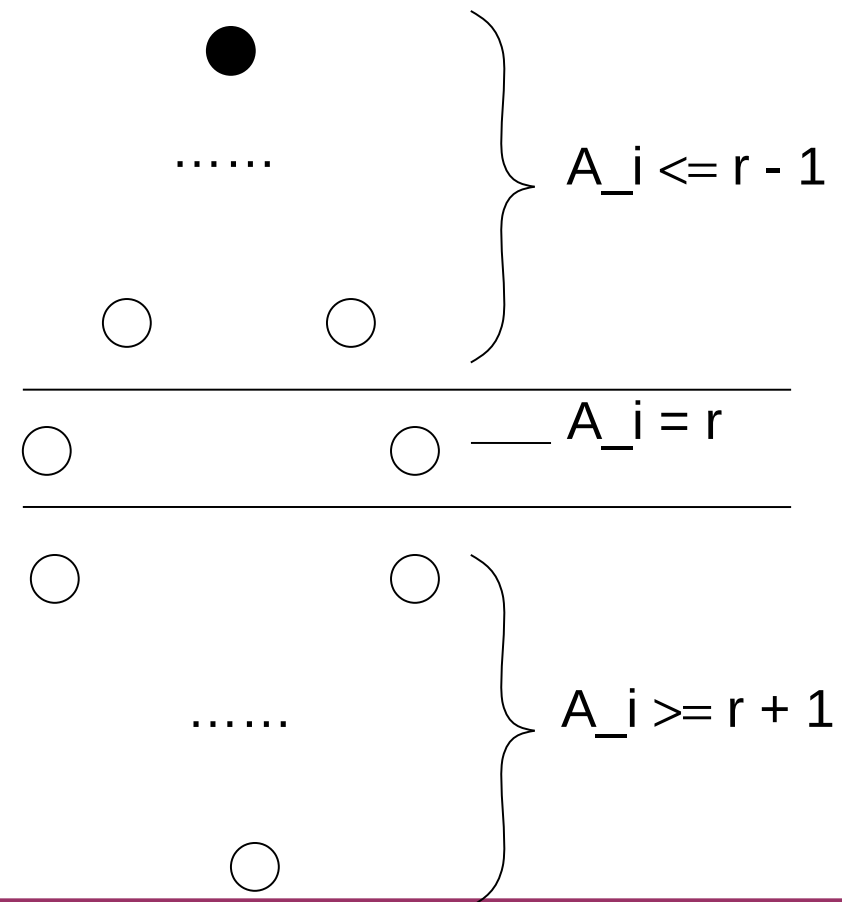
for nodes with	already know: phase r	want to show: phase $r+1$
$A_i \leq r-1$	$\text{dist}_i = A_i$	$\text{dist}_i = A_i$
$A_i = r$	$\text{dist}_i \geq r$	$\text{dist}_i = A_i$
$A_i \geq r+1$	$\text{dist}_i \geq r$	$\text{dist}_i \geq r+1$



Claim 2: Each node i will satisfy the red conditions at some point of time during phase $r+1$.

Proof: By definition of a phase, node i takes at least one action in phase $r+1$. Consider three possible cases for node i : $A_i \leq r-1$, $A_i = r$ and $A_i \geq r+1$. Also, leverage the fact the blue conditions always hold throughout phase $r+1$.

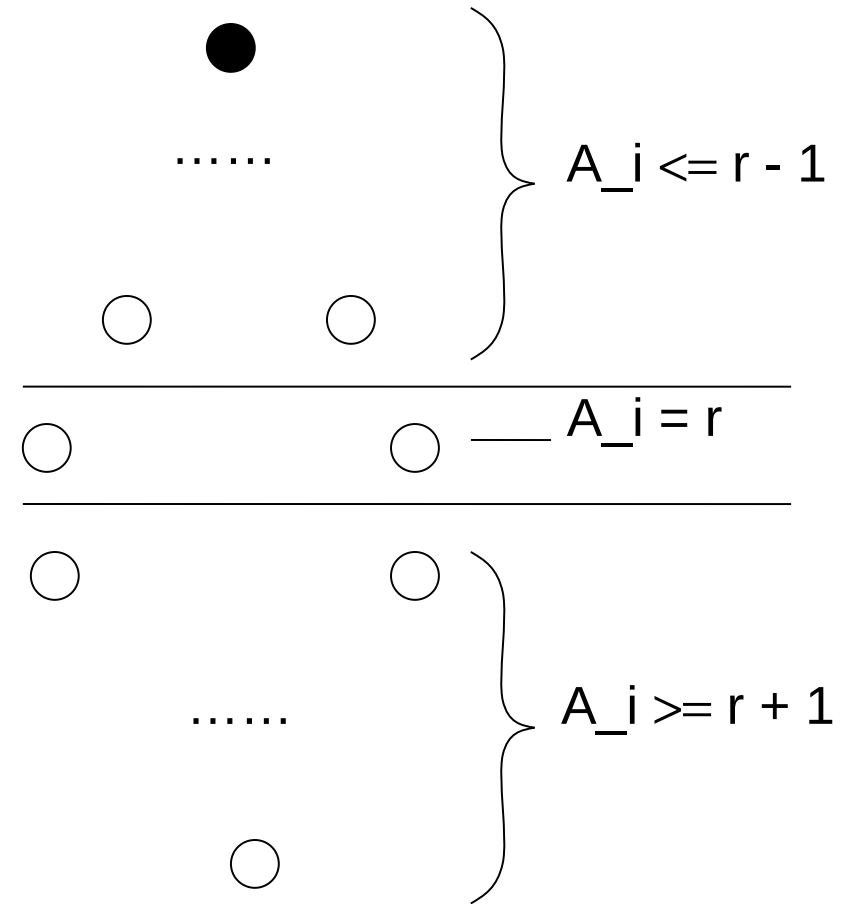
for nodes with	already know: phase r	want to show: phase $r+1$
$A_i \leq r-1$	$\text{dist}_i = A_i$	$\text{dist}_i = A_i$
$A_i = r$	$\text{dist}_i \geq r$	$\text{dist}_i = A_i$
$A_i \geq r+1$	$\text{dist}_i \geq r$	$\text{dist}_i \geq r+1$



Claim 3: For each node after it first satisfies the red condition, it will continue to satisfy the red condition for the remainder of phase $r+1$.

Proof: Enumerate three cases, and also to leverage the fact that the blue conditions always hold throughout phase $r+1$.

for nodes with	already know: phase r	want to show: phase $r+1$
$A_i \leq r-1$	$\text{dist}_i = A_i$	$\text{dist}_i = A_i$
$A_i = r$	$\text{dist}_i \geq r$	$\text{dist}_i = A_i$
$A_i \geq r+1$	$\text{dist}_i \geq r$	$\text{dist}_i \geq r+1$



Correctness Proof

- We just finished the inductive proof for the following lemma...
- Lemma: At the end of phase r ,
 - Any process i whose $A_i \leq r-1$, we have $\text{dist}_i = A_i$;
 - Any process i whose $A_i \geq r$, we have $\text{dist}_i \geq r$;

Correctness Proof

- Theorem: After $H+1$ phases, $dist_i = A_i$ on all processes
 - H being the length of the shortest path from the most far away process to the root
 - Directly follows from the earlier lemma.
- Theorem: After $H+1$ phases, the *dist* and *parent* values on all processes are correct
 - Proof: Each process has a single parent pointer except the root. So the graph has n nodes and $n-1$ edges. Each process has a path to the root, thus the graph is connected. Hence it is a spanning tree.

Summary

- “Self-Stabilization”
- The concept of self-stabilization
- Self-Stabilizing Spanning Tree Construction
 - Correctness proof